Report On The Project-

Vehicle Maintenance Log

- **Submitted by -**

  1) Atif-Ul-Quadir, ID-2533574042
  2) Amman Latif Opy, ID-2411360042

- **Course - CSE215, Programming Language II**
- **Section- 11**
- **Submitted To - Dr. Shamim Al Mamun**
- **Submission Date - 16/08/2026**

1. Introduction

The **Vehicle Maintenance Log** is a Java-based application designed to help users keep track of vehicle maintenance and registration information. The system allows users to check mileage-based maintenance such as engine oil, brake fluid, transmission fluid, and tire changes. It also checks whether a vehicle's registration renewal is due.

The project was developed using Java and a graphical user interface using **Java Swing**. The main purpose of the project is to demonstrate the use of **inheritance, polymorphism, abstraction, interfaces, and Swing** in a practical application.

---

2. Objectives

The main objectives of the project are:

- To develop a simple vehicle maintenance tracking system.
- To apply inheritance between related classes.
- To demonstrate abstraction using an abstract class.
- To demonstrate polymorphism using parent-class references.
- To use an interface for common maintenance functionality.
- To create a graphical user interface using Java Swing.
- To check whether different maintenance tasks are due.

---

3. Technologies Used

The project was developed using:

- Java
- Java Swing

- Java AWT
- `LocalDate`
- Object-Oriented Programming concepts

---

4. Class Structure

The project consists of the following main classes:

vehicleDet

The `vehicleDet` class stores the basic information of a vehicle, including its model, brand, registration date, and tires. It also contains constructors and getter/setter methods.

MaintenanceTask

`MaintenanceTask` is an abstract class that provides the common structure for maintenance tasks. It extends `vehicleDet` and implements the `Serviceable` interface. It also contains the abstract `isDue()` method.

MileageTask

`MileageTask` is a subclass of `MaintenanceTask`. It handles maintenance that depends on the vehicle's mileage, such as oil and fluid changes.

TimeTask

`TimeTask` is another subclass of `MaintenanceTask`. It is used to determine whether vehicle registration renewal is required based on the date.

Serviceable

`Serviceable` is an interface that defines the `resetCounter()` method.

Swing

The `Swing` class contains the main program and creates the graphical user interface, vehicle objects, maintenance tasks, table, buttons, and event handling.

---

5. Abstraction

Abstraction is implemented using the `MaintenanceTask` abstract class.

The class contains an abstract method called `isDue()`. Since the method does not have a common implementation, each subclass determines how it should check whether a task is due.

This is useful because mileage-based maintenance and time-based maintenance require different calculations.

For example:

- `MileageTask` checks the vehicle's mileage.
- `TimeTask` checks the registration date.

Therefore, the common maintenance structure is placed in the abstract class while the specific behavior is handled by the subclasses.

---

6. Inheritance

Inheritance is used to create a relationship between the vehicle and maintenance classes.

The basic structure is:

vehicleDet → MaintenanceTask → MileageTask / TimeTask

`MaintenanceTask` inherits vehicle-related information from `vehicleDet`. `MileageTask` and `TimeTask` then inherit the common maintenance functionality from `MaintenanceTask`.

This reduces the need to repeat common properties and methods in every maintenance class.

---

7. Polymorphism

Polymorphism is demonstrated by storing different child objects inside a `MaintenanceTask` array.

For example, the program creates an array of `MaintenanceTask` objects and stores both MileageTask and TimeTask objects in it.

This allows the program to treat different maintenance tasks as `MaintenanceTask` objects while still using their individual implementations.

For example, when `isDue()` is called, `MileageTask` performs a mileage calculation, while TimeTask performs a date calculation.
This is an example of **runtime polymorphism**.

---

8. Interface

The project uses the `Serviceable` interface to define a common operation for maintenance tasks.

The interface contains a `resetCounter()` method.

The subclasses implement this method differently.

For `MileageTask`, resetting means updating the previous maintenance mileage to the current mileage.

For `TimeTask`, resetting means updating the registration date to the current date.

Thus, the interface provides a common rule while allowing each class to implement that rule differently.

---

9. Swing GUI

Java Swing is used to create the graphical interface of the application.

The main window displays vehicle information using a `JTable`. The table contains information such as:

- Brand
- Car Model
- Last Updated Odometer Reading
- Registration Date
- Tires

The application provides two main buttons:

- **Check for Oil and Fluid Changes**
- **Check for Registration Renewal**

The user first selects a vehicle from the table. The application then performs the selected operation.

---

10. Maintenance Checking

When the user checks for oil and fluid maintenance, the program asks for the vehicle's current odometer reading.

The mileage is then compared with the previous maintenance mileage and the predefined maintenance interval.

If the required interval has been reached, the task is displayed as **DUE**. Otherwise, it is displayed as **NOT DUE**.

The application also provides an Update button for each maintenance task. When the task is updated, its maintenance counter is reset.

The system supports several mileage-based tasks, including:

- Engine Oil Change
- Brake Fluid Change
- Transmission Fluid Change
- Tire Change

---

11. Registration Renewal

The application also checks whether the vehicle registration needs to be renewed.

The `TimeTask` class compares the current date with the stored registration date. If more than one year has passed, the registration is considered due.

If renewal is due, the program asks the user whether they want to renew it. If the user confirms, the registration date is updated and the information in the JTable is refreshed.

---

12. Working Process

The general working process of the application is:

1. The application starts and displays the main Swing window.
2. Vehicle information is displayed in a table.
3. The user selects a vehicle.
4. The user chooses either maintenance checking or registration checking.

5. For maintenance, the current mileage is entered.
6. The system determines whether each maintenance task is due.
7. The user can update completed maintenance.
8. For registration, the system checks the registration date.
9. If renewal is required, the user can renew the registration.
10. The updated information is displayed in the GUI.

---

13. Advantages

The system provides the following advantages:

- Simple graphical interface.
- Easy vehicle selection.
- Separate mileage-based and time-based maintenance.
- Automatic maintenance status checking.
- Individual maintenance updates.
- Demonstrates important Object-Oriented Programming concepts.
- Uses polymorphism to handle different maintenance task types.

---

14. Limitations

The current project is designed primarily as an academic demonstration of Java OOP concepts.

The vehicle information is initialized within the program, so the system does not permanently store information after the application closes. The number of vehicles and maintenance tasks is also fixed.

These limitations are acceptable for the project's purpose, which is to demonstrate **abstraction, inheritance, polymorphism, interfaces, and Swing**.

---

15. Conclusion

The **Vehicle Maintenance Log** demonstrates how the major Object-Oriented Programming concepts can be combined to create a functional Java application.

Abstraction is implemented through the MaintenanceTask abstract class. **Inheritance** connects vehicleDet, MaintenanceTask, MileageTask, and TimeTask. **Polymorphism** allows different maintenance task objects to be handled through common MaintenanceTask references. The **Serviceable interface** provides a common method for resetting maintenance information, while each task implements it according to its own requirements.

Finally, **Java Swing** provides the graphical interface through which users can select vehicles, check maintenance requirements, update maintenance records, and check registration renewal.

The project therefore provides a practical demonstration of the required Java concepts while solving a simple real-world vehicle maintenance problem.